

Phase 1 Proposal — NYC 311 Service Request Analytics

Team members: Noyan Siddiqui (24L-2593), Abdul Rehman (24L-2613)

Title: New York City 311 Service Request Analytics

1. Domain and Source Identification

Project Concept

Our project focuses on public services and on how a large city responds to its residents. Every day around ten thousand people call 311, the non emergency complaint line of New York City, about potholes, broken heating, noise, illegal parking and similar problems. Each call becomes a service request which is sent to a city agency and closed once it is resolved. The city publishes all of these requests as open data, with more than forty million records since 2010, updated every day.

The main question we want to answer is how quickly the city responds when its residents ask for help, and whether the answer depends on which neighbourhood they live in. We will also study which agencies handle the heaviest workload, which complaint types are growing, and how the seasons change what people complain about.

Data Source

Our data source is the NYC Open Data portal, the official open data platform of the City of New York. The dataset is called [311 Service Requests from 2020 to Present](#).

This data is not published as reports or PDF documents. It is published through a web API called the Socrata Open Data API, so we request records directly from a web address and receive them in CSV format. No API key is needed, although we will register a free application token so the portal does not slow our requests during the historical load.

The data dictionary records three limitations we will state in our final report. The dataset excludes agencies that run their own customer service systems, notably the New York City Housing Authority and the Department of Corrections, so it is not a complete picture of every complaint in the city. The records represent complaints made about the city rather than an objective measurement of city conditions, which means a neighbourhood with many noise complaints is the neighbourhood that complains most, not necessarily the noisiest one. The Open Data Channel Type field may also be unreliable from March 2020 onwards because of changes the city made to call handling during the pandemic, which matters because our window begins in January 2020.

Data Extraction Method

Since our source is an API, there are no PDF files in this project and no tables have to be extracted from documents. We are not using any PDF extraction library such as pdfplumber or camelot because there is nothing to parse.

The tools we will use are:

- Python with the requests library, to call the API
- pandas, to read the returned CSV into a dataframe
- PySpark inside Databricks, to convert that dataframe and write it to Bronze as Parquet

The dataset contains a single table in which each row is one service request, so there is no question of choosing which tables to extract. Working from the official data dictionary, the fields we will extract are:

- Unique Key, the identifier of each service request
- Created Date, Closed Date, Due Date and Resolution Action Updated Date
- Agency and Agency Name
- Complaint Type, which the city has renamed Problem, and Descriptor, renamed Problem Detail, together with Additional Details
- Status and Resolution Description
- Location Type and Address Type
- Incident Zip, City, Borough, Community Board, Council District and Police Precinct
- Open Data Channel Type, which records whether the request came in by phone, online, mobile or another source
- Incident Address, Street Name, Cross Streets, BBL, Latitude and Longitude, which are collected into Bronze and then removed in Silver

Full Load Ingestion

For the full load we collect the complete historical baseline, pulling the records one year at a time so that each run produces a manageable file. The API lets us filter by created date and request a fixed number of rows at a time, so our script keeps asking for the next set of rows until nothing more is returned and then moves to the next year. This is a Python loop stored in our repository so that the load can be repeated and produce the same result.

Incremental Load Ingestion

The city publishes this dataset daily and the data dictionary confirms both the publishing frequency and the data change frequency as daily. The update runs overnight with a one day buffer, which means that a dataset viewed on Thursday morning contains everything

through Tuesday and the early hours of Wednesday. Our job therefore always asks for everything after the latest created date already in Bronze rather than assuming yesterday is available.

Two kinds of change occur every day. New requests arrive, between eight and ten thousand of them. Existing requests also change, because a request moves through statuses such as Assigned, Pending, Closed or Canceled, and through all of this its Unique Key stays the same while the Status, Closed Date, Resolution Description and Resolution Action Updated Date are rewritten.

Our incremental load is therefore a merge rather than an append. For every incoming record we check whether the identifier already exists, inserting it if it does not and updating it if it does. We will use the last updated system field provided by Socrata to catch records that changed without being newly created. If nothing is new on a given day the job records that and finishes. Once scheduled in Databricks it runs daily without us touching it.

2. Data Samples and Volume

Sample Files

We will provide two sample raw files in our repository, both of them real API output rather than files we made by hand:

- Full load sample, containing roughly fifty thousand rows taken from our historical collection
- Incremental load sample, containing one day of API output, meaning the requests created that day together with the older requests updated that day

We are not pushing the entire dataset to the repository, since a repository is meant for code and representative samples.

Expected Volume and Frequency

New York City receives roughly three million service requests each year, and our window runs from January 2020 to the present, which is about six and a half years. Our estimates for the full load are:

- Rows: approximately 20,000,000
- Columns: approximately 40 per row
- Average row size as CSV: approximately 400 bytes
- Extracted size as raw CSV: approximately 8 GB
- Stored size in Bronze as compressed Parquet: approximately 1 to 1.5 GB

Parquet is much smaller than CSV here because it stores data column by column and the values in this dataset repeat heavily.

Our estimates for the incremental load are:

- Frequency: daily, since the city refreshes the dataset every day
- New rows per day: approximately 8,000 to 10,000
- Updated rows per day: existing requests whose status or closed date changed
- Extracted size per day as CSV: approximately 4 MB
- Stored size per day as Parquet: approximately 1 MB

These figures are estimates of the extracted data and not of any source file. We will replace them with measured figures after the first backfill year, by reading the row count and stored size directly from the Bronze table. If 20,000,000 rows proves too large for the free tier we will reduce the window to three years, which is approximately 9,000,000 rows and 500 MB of Parquet.

3. Security and Compliance

This dataset does contain some personally identifiable information, so we are not claiming otherwise. The city states in its data dictionary that the data does not reveal any personally identifying information about the customer who made the request, and there are indeed no names, phone numbers or email addresses. However, the following sensitive fields remain:

- Incident address, street name and cross streets
- Latitude, longitude and the coordinate fields
- BBL, the borough block and lot parcel number, which identifies an individual building or property and can be joined to public property ownership records

An address combined with a complaint effectively says who complained about what, and in a small building that identifies a person.

The remaining fields are safe to keep:

- Zip code, borough and community board, which describe areas containing thousands of residents
- Complaint type, agency, dates and status, which contain no personal information at all

Our handling strategy is to drop the sensitive columns when the data moves from Bronze to Silver. We do not remove them at the point of collection because the purpose of Bronze is to hold the raw response exactly as received so that our work stays auditable. We drop rather than hash, because a hash of a limited and publicly known set of addresses can be

reversed easily and we do not need building level counts. We will also suppress any figure in the dashboard that falls below a small count threshold, since even area level data can identify a person when the area is thinly populated and the complaint is unusual.

4. High Level Medallion Data Modelling

Bronze Layer

Both the full load and the daily incremental load run through the same code once the data reaches Bronze. Only the list of records being requested differs.

The Bronze layer holds every API response exactly as received, with no changes, stored as Parquet and partitioned by ingestion date. Alongside the data we record the source address called, the ingestion time, a batch identifier, whether the batch was a full or incremental load, and the number of rows returned. Nothing is cleaned here, so if the city changes a column name the Bronze layer captures that faithfully and Silver deals with it.

Silver Layer

The Silver layer is where the data is cleaned and made consistent. The processing steps are:

- Casting Created Date, Closed Date, Due Date and Resolution Action Updated Date from text into proper timestamps, since the dictionary gives them as MM/DD/YY HH:MM:SS AM/PM
- Keeping rows with an empty Closed Date, because that means the request is still open and is valid data rather than an error
- Standardising the complaint hierarchy, which the city renamed in 2025 from Complaint Type and Descriptor to Problem and Problem Detail, so that older and newer records use one agreed set of names
- Mapping every raw complaint value through a lookup table stored in our repository, and failing the job when a value is missing from the lookup instead of silently dropping the row
- Deduplicating on Unique Key, keeping the most recently updated version
- Dropping the sensitive columns listed in the previous section
- Deriving the resolution time as the difference between Created Date and Closed Date
- Normalising Status, whose values include Assigned, Pending, Closed and Canceled, and deciding explicitly how Canceled requests are treated rather than letting them count as resolved
- Normalising Borough, whose spelling and capitalisation vary, and handling the Unspecified value deliberately

The result is a single clean table with one row per service request, holding the identifier, the created and closed timestamps, the agency code and name, the standardised complaint type, the descriptor, the status, the borough, the zip code, the community board, the channel, the resolution time, a flag for whether the request is still open, and the lineage columns carried down from Bronze.

Gold Layer

The Gold layer holds summarised data that can be used directly for analysis and for the dashboard, so the raw data does not have to be reprocessed for every question. We model it as a star schema.

The fact table holds one row per service request with its created and closed timestamps, resolution time in hours, status and open flag. Around it sit four dimension tables:

- Agency dimension, holding each city agency with its code and full name
- Complaint dimension, holding each standardised complaint type with a broader category such as noise, housing, street or sanitation
- Geography dimension, holding incident zip, community board, council district, police precinct and borough
- Date dimension, holding year, month, quarter, day of week and season

On top of the star schema we will build summary tables so the dashboard loads quickly:

- Median and ninetieth percentile resolution time per agency per month, with request volume
- Request counts by complaint category, borough and month
- Median resolution time for the same complaint type across boroughs, which our central question depends on
- Daily and monthly request counts per complaint category, for seasonality
- Count of requests still open, grouped by how old they are and which agency holds them

We report the median rather than the average because resolution times have a long tail, and we show the ninetieth percentile beside it so that the tail stays visible.

5. Business Intelligence and Dashboard

The dashboard will be built in Power BI Desktop reading the Gold tables directly. Its purpose is to show how quickly the city responds to its residents and whether that response is consistent across different parts of the city.

The business questions the dashboard will answer are:

- Which agencies are fastest and slowest at closing requests, and has that changed over time
- For the same complaint type, does resolution time differ between boroughs
- What do residents complain about, and how does that change with the seasons
- Which areas have the highest request volume and the slowest resolution
- How large is the backlog of open requests, and which agency is carrying most of it

The visual charts we plan to deliver are:

- Line chart of agency performance over time, plotting median resolution hours per month with one line per agency, filtered by complaint category and borough
- Grouped bar chart showing median resolution time per borough for a single selected complaint type, with request volume on a second axis so a small sample is not mistaken for a real difference
- Heatmap of complaint categories against months, coloured by request volume
- KPI cards for total requests, median resolution time, percentage still open and average daily volume
- Map of the city shaded by median resolution time in each zip code

6. Infrastructure and Cost Management

We plan to use Databricks Free Edition for processing with Apache Spark and Power BI Desktop for the dashboard. Data is stored as Parquet partitioned by year and month, and the daily job is scheduled using Databricks Jobs.

Our GitHub repository will be public from the day it is created. It will hold the two ingestion scripts, the notebooks for the Bronze, Silver and Gold layers, the complaint type lookup table, the two sample data files and a README explaining how to run the pipeline.

Databricks Free Edition gives limited compute, so four things keep us inside it:

- We develop all transformation logic against a single month of data, and only run the full history once the code is proven
- We can narrow the window from six years to three if the volume proves too large
- We store everything as Parquet after landing, so a query filtered to one year never reads the rest
- The incremental job handles only about ten thousand rows a day, which is seconds of compute rather than minutes

We will also register a free Socrata application token, because without one the portal limits our request rate and a backfill of several hundred requests would stall partway through.